Edge Vision: Real-Time Skeleton-Based Violence Detection using ST-GCN and Multi-Agent Kalman Tracking

Parth Aishpunani

Apex Institute of Technology
Chandigarh University, Mohali, Punjab, India

Abstract*—Real-time automated violence detection across distributed surveillance cameras is constrained by hardware compute ceilings, high false-positive rates on benign motion, and privacy hazards when raw video is streamed to centralized servers. Traditional 3D convolutional and video-transformer architectures operate over dense RGB pixel volumes, yielding large memory footprints and brittle sensitivity to background clutter, illumination shifts, and camera jitter. This work introduces Edge Vision, a dual-tier real-time violence detection platform unifying lightweight pose estimation (YOLOv8n-Pose), a Spatial-Temporal Graph Convolutional Network (ST-GCN), an 8-state constant-velocity Kalman occlusion tracker with Hungarian bipartite assignment, and a novel pixel-space geometric Relational Strike Gate. Representing subjects via normalized 17-node COCO skeletal graphs across 30-frame temporal windows compresses per-frame input bandwidth by 99.8% while decoupling biomechanical motion from appearance confounders. The identical ONNX graph pipeline runs inside both a memory-bounded FastAPI cloud microservice (<512 MB RAM ceiling) and a zero-upload client browser engine accelerated via WebGPU and COOP/COEP-isolated multi-threaded WebAssembly (ONNX Runtime Web). Benchmarked over 100 timed iterations (after a 10-iteration warm-up) on an Intel Core i7-12700H CPU, the pipeline executes at 50.7 FPS end-to-end (16.9 ms summed model latency). On a strictly held-out, zero-leakage chronological test split of 293 unseen videos partitioned from a 1,951-video benchmark corpus (TP=142, FP=34, FN=8, TN=109), the complete system achieves 85.67% Accuracy, 80.68% Precision, 94.67% Recall, 76.22% Specificity, and an F1 Score of 0.871.

***Index Terms*—Violence detection, spatial-temporal graph convolutional networks, pose estimation, Kalman filtering, Hungarian algorithm, relational strike gating, WebAssembly, edge AI.**

I. INTRODUCTION

Automated surveillance systems deployed in public transit corridors, campuses, correctional facilities, and retail spaces require sub-second detection of physical altercations while minimizing operator alert fatigue. Transmitting continuous high-definition video streams to centralized GPU clusters incurs prohibitive network egress costs, variable transmission latency, and regulatory compliance risks when personally identifiable visual frames leave local premises. Conversely, deploying heavy deep learning detectors directly onto low-power edge nodes, browser sandboxes, or memory-capped cloud micro-containers imposes strict limits on RAM consumption, binary size, and single-frame compute latency.

First-generation deep learning violence detectors processed volumetric RGB pixel tensors directly using 2D+LSTM hybrids or 3D Convolutional Neural Networks (3D-CNNs) such as C3D [1]. Although effective on curated indoor clips, dense pixel models exhibit fundamental operational drawbacks: (i) cubic memory scaling over spatial height, spatial width, and temporal depth, rendering real-time inference infeasible without high-wattage discrete GPUs; and (ii) severe confounding by nuisance visual factors including lighting flicker, rain, camera vibration, and clothing color.

Skeleton-based action recognition mitigates both challenges by projecting each human subject into a compact topological graph of anatomical keypoints [2]. Compressing 1920×1080×3 image rasters into normalized (17, 2) coordinate pairs strips background texture and personally identifiable facial detail prior to sequence modeling. However, standalone skeletal graph classifiers remain vulnerable to two failure modes in unconstrained surveillance feeds: (1) track fragmentation and identity swaps during physical grapples where combatants heavily occlude one another, and (2) elevated false-positive rates caused by rapid isolated movements—such as dancing, vigorous gesturing, or solo jumping—that exhibit high joint velocity without hostile inter-actor contact.

To resolve these failure modes while preserving real-time CPU throughput, we present **Edge Vision**, making four concrete technical contributions:

1. **Unified Missing-Joint Masked ST-GCN Pipeline:** A lightweight residual Spatial-Temporal Graph Convolutional Network exported as a single self-contained 257 KB ONNX binary paired with mid-hip root centering, torso scaling, and strict [0.0, 0.0] validity masking for occluded joints.
2. **True 8-D Kalman + Hungarian Occlusion Tracker:** Analytic constant-velocity Kalman state prediction and optimal IoU linear assignment maintaining subject identity across up to 10 consecutive occluded frames during clinches and takedowns.
3. **Pixel-Space Relational Strike Gating:** A deterministic two-invariant geometric filter measuring minimum Euclidean effector-to-target-zone proximity (d_{strike} in pixels) alongside bounding-box overlap (IoU > 0),

contrasted explicitly against naive box-centroid distance heuristics.

4. **Zero-Upload Cross-Origin-Isolated Edge + Cloud Architecture:** Bit-aligned inference running identically inside a 512 MB FastAPI server and a browser client unlocked with `COOP/COEP SharedArrayBuffer` multi-threaded WebAssembly SIMD and WebGPU acceleration.

II. RELATED WORK

Research in automated violence detection and skeletal kinematics has evolved rapidly between 2020 and 2026 across four intersecting subfields.

A. RGB-Volumetric and Weakly Supervised Violence Detection

Large-scale benchmarks catalyzed empirical study of unconstrained surveillance altercations. Soliman et al. [5] curated the *Real Life Violence Situations* dataset capturing diverse indoor and outdoor fights versus benign daily motions. Cheng et al. [6] introduced RWF-2000 from public surveillance feeds combining RGB and optical flow streams. To scale weak supervision, Wu et al. [7] released the multimodal XD-Violence corpus using audio-visual fusion branches, while Degardin and Proença [8] introduced UBI-Fights providing frame-level anomaly annotations across 1,000 sequences. Gohar [9] evaluated Convolutional Block Attention Modules (CBAM) combined with ConvLSTM networks on UBI-Fights to isolate fight regions from background motion. Despite strong accuracy, RGB and optical-flow volumetric networks retain large memory footprints and struggle under strict CPU-only edge latency limits.

B. Skeleton-Based Action Recognition via Graph Convolutions

Yan et al. [2] pioneered Spatial-Temporal Graph Convolutional Networks (ST-GCN), establishing normalized anatomical adjacency matrices coupled with 1D temporal kernels. Subsequent work expanded receptive fields and adaptive topology learning: Shi et al. [10] introduced Two-Stream Adaptive GCN (2s-AGCN) to learn data-driven graph edges alongside bone vectors; Liu et al. [11] proposed MS-G3D to disentangle multi-scale neighborhood hops and cross-spacetime skip connections; and Chen et al. [12] formulated CTR-GCN to refine graph topology independently across feature channels.

To lower inference overhead, Song et al. [13] designed EfficientGCN using compound scaling and separable convolutions. Duan et al. [14] revisited skeleton pipelines via PoseConv3D over stacked 3D joint heatmaps and further standardized ST-GCN++ best practices in the PYSKL benchmark [15]. Chi et al. [16] introduced InfoGCN employing information-bottleneck objectives and self-attention graph modules. More recently, Lee et al. [17] proposed Hierarchically Decomposed GCN (HD-GCN) to connect semantically distant joint groups, Do and Kim [18] introduced SkateFormer partitioning skeletal-temporal transformers across joint and frame axes, and Zhou et al. [19] developed BlockGCN preserving topology awareness. While highly expressive for general action classes, dense multi-stream and transformer architectures exceed real-time WebAssembly latency budgets in browser runtimes.

C. Skeleton Violence Detection and Relational Gating Contrast

Applying skeletal graphs directly to violent altercation detection eliminates environmental pixel bias. Yang et al. [20] designed multi-scale attention-enhanced GCNs fusing joint positions, bone vectors, and motion differentials for skeleton-based violence recognition. Janbi et al. [21] proposed ESTS-GCN, ensembling spatial-temporal skeleton topologies to capture physical brawls. Tran et al. [22] investigated interaction-line graph representations (SKE-A3TGCN and GCNVertexEdge) modeling pairwise inter-person joint relations for low-resolution CCTV feeds.

A key gap in prior skeleton violence pipelines is handling high-energy non-contact gestures. Standard proximity heuristics threshold only *bounding-box centroid distance* $\|\mathbf{c}_A - \mathbf{c}_B\|_2 < d_{\text{thresh}}$. Centroid distance fails systematically under two common geometries: (i) two tall standing actors side-by-side in peaceful conversation may exhibit small centroid spacing while their hands hang stationary at their sides; conversely, (ii) two combatants executing a wide flying kick or extended punch maintain large torso-centroid separation while an extremity (wrist or ankle) directly strikes the opponent's head or torso. Edge Vision's novel **Relational Strike Gate** resolves this directly in pixel space by pairing extremal strike effectors {9, 10, 15, 16} against opponent vulnerable target zones {0..12} and box extents alongside bounding-box convergence (IoU > 0), decoupling strike contact geometry from center-of-mass spacing.

D. Multi-Object Tracking and Edge Runtime Optimization

Bottom-up and single-stage pose detectors such as YOLO-Pose [23] and Ultralytics YOLOv8n-Pose [3] provide simultaneous person localization and COCO-17 regression within a single forward pass. For temporal identity stabilization, modern tracking-by-detection methods—including ByteTrack [24] and BoT-SORT [25]—combine linear Kalman filters with Kuhn's Hungarian assignment solver [4]. ByteTrack proved that geometric IoU association with low-confidence box retention outperforms heavy CNN Re-ID embeddings on compute-constrained hardware. For runtime portability, ONNX Runtime [26] unifies graph optimizations across native C++ server threads and browser WebGPU/WASM execution providers, while MobileNetV3 [27] provides low-overhead scene context classification for athletic context attenuation.

III. PROPOSED METHODOLOGY

Figure 1 illustrates the sequential Edge Vision inference pipeline shared across the Python backend and browser

WebAssembly runtime.

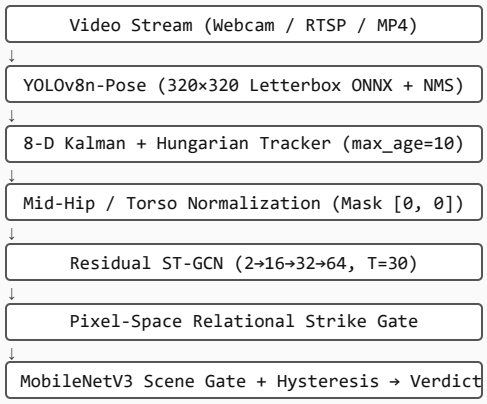


Fig. 1. Unified End-to-End Pose, Kalman Tracking, ST-GCN, and Relational Strike Gate Architecture.

A. Pose Extraction and Coordinate Normalization

Incoming video frames of native pixel resolution $W_{\text{frame}} \times H_{\text{frame}}$ are processed at stride s ($s = 2$ for continuous streaming/benchmarks, $s = 1$ for sequentially subsampled browser clips). Each sampled frame is aspect-preservingly letterboxed into a 320×320 canvas with RGB constant padding (114, 114, 114) via uniform scaling factor $\gamma = \min(320/W_{\text{frame}}, 320/H_{\text{frame}})$ and integer offsets (p_x, p_y). After stripping any browser canvas alpha channel, the tensor is transposed to NCHW RGB order in [0.0, 1.0] and evaluated by YOLOv8n-Pose, returning raw anchors $\mathbf{O} \in \mathbb{R}^{1\times 56\times 2100}$ encoding $[c_x, c_y, w, h, \text{conf}, \{x_k, y_k, c_k\}]$. Candidate boxes undergo greedy IoU Non-Maximum Suppression ($\tau_{\text{NMS}} = 0.45$) and map strictly back to un-normalized **pixel-space coordinates** on $[0, W_{\text{frame}}-1] \times [0, H_{\text{frame}}-1]$:

$$x_{\text{px}} = \text{clip}\left(\frac{x_{\text{model}} - p_x}{\gamma}, 0, W_{\text{frame}}-1\right), \quad y_{\text{px}} = \text{clip}\left(\frac{y_{\text{model}} - p_y}{\gamma}, 0, H_{\text{frame}}-1\right) \tag{1}$$

Prior to ST-GCN sequence formatting, each primary actor's (17, 2) coordinates are first divided by $(W_{\text{frame}}, H_{\text{frame}})$ into $[0, 1]^2$ frame coordinates $\mathbf{p}_k$ and masked via indicator $m_k = \mathbb{I}(\mathbf{p}_k \neq (0, 0))$. Mid-hip root $\mathbf{m}_{\text{hip}}$ (from valid hips {11, 12}) and torso scale L_{torso} are computed solely over valid joints ($m_k = 1$):

$$\hat{\mathbf{p}}_k = \frac{\mathbf{p}_k - \mathbf{m}_{\text{hip}}}{\max(L_{\text{torso}}, 0.05)} \ (\text{if } m_k{=}1, L_{\text{torso}}{>}0.01); \ \text{else } (0.0, 0.0) \tag{2}$$

Strictly enforcing $\hat{\mathbf{p}}_k = (0.0, 0.0)$ when $m_k = 0$ prevents synthetic coordinate drift on missing joints.

B. Multi-Agent Kalman Tracking and Hungarian Association

To prevent identity switching during grapples and transient occlusions, `OcclusionKalmanTracker` tracks each bounding box via an 8-dimensional constant-velocity Kalman state vector $\mathbf{x}_k \in \mathbb{R}^8$ in pixel units:

$$\mathbf{x}_{k|k} = [\, c_x, c_y, w, h, v_x, v_y, v_w, v_h \,]^{\text{T}} \tag{3}$$

State transition $\mathbf{F} \in \mathbb{R}^{8\times 8}$ and observation matrix $\mathbf{H} \in \mathbb{R}^{4\times 8}$ are defined as:

$$\mathbf{F} = \begin{vmatrix} \mathbf{I}_4 & \mathbf{I}_4 \\ \mathbf{0}_4 & \mathbf{I}_4 \end{vmatrix}, \quad \mathbf{H} = [\, \mathbf{I}_4 \ \ \mathbf{0}_4 \,] \tag{4}$$

Covariance initializes as $\mathbf{P}_0 = \text{diag}(10\mathbf{1}_4, 1000\mathbf{1}_4)$, process noise $\mathbf{Q} = \text{diag}(\mathbf{1}_4, 0.01\mathbf{1}_4)$, and sensor noise $\mathbf{R} = \text{diag}(1, 1, 10, 10)$.

1) Prediction Phase

At discrete step k, prior state and covariance advance analytically via four 4×4 block updates without dense 8×8 multiplications:

$$\mathbf{x}_{k|k-1} = \mathbf{F}\mathbf{x}_{k-1|k-1}, \quad \mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1|k-1}\mathbf{F}^{\text{T}} + \mathbf{Q} \tag{5}$$

2) Hungarian Linear Assignment

Predicted boxes $\{\hat{\mathbf{b}}_i\}$ and YOLOv8 detections $\{\mathbf{z}_j\}$ undergo optimal bipartite assignment [4] minimizing cost $C_{ij} = 1 - \text{IoU}(\hat{\mathbf{b}}_i, \mathbf{z}_j)$ via `scipy.optimize.linear_sum_assignment` subject to IoU ≥ 0.15; unmatched tracks persist up to `max_age = 10` frames.

3) Measurement Update Phase

Matched tracks correct predicted state with box observation $\mathbf{z}_k = [c_x^{\text{det}}, c_y^{\text{det}}, w^{\text{det}}, h^{\text{det}}]^{\text{T}}$ using linear system solve `np.linalg.solve` for Kalman gain $\mathbf{K}_k$:

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H}\mathbf{x}_{k|k-1}, \quad \mathbf{S}_k = \mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^{\text{T}} + \mathbf{R} \tag{6}$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}^{\text{T}}\mathbf{S}_k^{-1}, \quad \mathbf{x}_{k|k} = \mathbf{x}_{k|k-1} + \mathbf{K}_k\mathbf{y}_k, \quad \mathbf{P}_{k|k} = (\mathbf{I}_8 - \mathbf{K}_k\mathbf{H})\mathbf{P}_{k|k-1} \tag{7}$$

C. Spatial-Temporal Graph Convolutional Networks (ST-GCN)

Normalized poses populate a rolling 75-pose buffer sampled uniformly (`linspace`) over $T = 30$ indices to assemble $\mathbf{X} \in \mathbb{R}^{1\times 2\times 30\times 17}$. The COCO-17 topology $G = (V, E)$ connects 17 joints along 16 anatomical bones encoded in symmetric adjacency $\mathbf{A} \in \{0, 1\}^{17\times 17}$ with normalized propagation matrix:

$$\tilde{\mathbf{A}} = \mathbf{D}^{-1/2}(\mathbf{A} + \mathbf{I}_{17})\mathbf{D}^{-1/2}, \quad D_{ii} = \Sigma_j(A_{ij} + \delta_{ij}) \tag{8}$$

Following `BatchNorm2d(2)`, three residual `STGCNBlock` layers expand channels across 2 → 16 → 32 → 64. Each block fuses spatial graph convolution ($\mathbf{X}\tilde{\mathbf{A}}$ followed by 1×1 Conv2d) and a 9×1 temporal convolution (stride (1, 1), padding (4, 0), dropout 0.2):

$$\mathbf{H}^{(l+1)} = \mathrm{Drop}(\sigma(\mathrm{BN}_t(\mathrm{Conv}_{9\times1}(\sigma(\mathrm{BN}_s(\tilde{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)})))) + \mathbf{R}^{(l)})) \quad (9)$$

Global average pooling over (T, V) maps to a linear head and runtime logit calibration σ(logit + 0.55).

D. Training Protocol & Reproducibility Details

All experiments enforce deterministic seeding (`seed=42`) across Python, NumPy, and PyTorch. To eliminate cross-window data leakage, `video_ids.json` binds every sample window strictly to its source clip ID, and `splits_manifest.json` enforces disjoint splits. Optimization uses `AdamW` (lr = 10^{-3}, weight decay = 10^{-4}), `CosineAnnealingLR` ($T_{\max}$ = 25, $\eta_{\min} = 10^{-5}$), gradient clipping (≤ 2.0), and `BCEWithLogitsLoss` (`pos_weight=1.3`) with 0.05 label smoothing. Three biomechanical augmentations (p = 0.5 each) apply during training: horizontal flip with bilateral joint swap, Gaussian noise $\mathcal{N}(0, 0.015^2)$, and uniform scale jitter [0.95, 1.05]. Specifications are summarized in Table I.

TABLE I
ST-GCN ARCHITECTURE & TRAINING REPRODUCIBILITY SPECIFICATIONS

Hyperparameter / Component	**Exact Specification**
Input Tensor Shape	(B, C=2, T=30, V=17)
ST-GCN Channels	2 → 16 → 32 → 64 (3 residual blocks)
Temporal Conv / Stride / Pad	(9, 1) / (1, 1) / (4, 0)
Dropout / Classifier Head	0.2 per block / GAP(T,V) → Linear(64→1)
Optimizer & LR Scheduler	AdamW (10^{-3}, wd 10^{-4}), CosineAnnealingLR
Loss / Smoothing / Pos Weight	BCEWithLogitsLoss (1.3 pos weight, 0.05 smooth)
Batch Size / Epochs / Clip	32 / 25 epochs / max_norm=2.0
ONNX Export Binary	Single embedded file (257 KB, opset 18)

E. Pixel-Space Relational Strike Gate & Unit Consistency

For frames of pixel width W_{frame} containing $N \geq 2$ actors, all keypoints $\mathbf{p}_i$ and boxes $\mathbf{b}_v$ are evaluated strictly in **identical pixel-space coordinates** on [0, W_{frame}−1] × [0, H_{frame}−1] (browser TypeScript scales [0, 1]^2 points back by (W_{frame}, H_{frame}) beforehand). Let strike effectors be S = {9, 10, 15, 16} (wrists, ankles) and vulnerable target joints be T = {0..12}. Clamped point-to-box distance in pixels is:

$$\mathrm{dist}_{\text{px}}(\mathbf{p}_i, \mathbf{b}_v) = \sqrt{(\max(b_{x1}-x_i, 0, x_i-b_{x2})^2 + \max(b_{y1}-y_i, 0, y_i-b_{y2})^2)} \quad (10)$$

Symmetric bidirectional strike distance (in pixels) is:

$$d_{\text{strike}}(A, B) = \min_{(u,v)\in\{(A,B),(B,A)\},\, i\in S_u, j\in T_v} \min(\|\mathbf{p}_i - \mathbf{p}_j\|_2, \mathrm{dist}_{\text{px}}(\mathbf{p}_i, \mathbf{b}_v)) \quad (11)$$

Any alert above 0.65 clamps to 0.25 unless at least one pair satisfies both invariants in matching pixel units:

$$d_{\text{strike}}(A, B) \leq \alpha \cdot W_{\text{frame}} \quad \text{AND} \quad \mathrm{IoU}(\mathbf{b}_A, \mathbf{b}_B) > 0.0 \quad (12)$$

where α = 0.22 in standard surveillance (α = 0.12 and confirmation threshold τ = 0.85 when 1 FPS MobileNetV3 scene gating flags sports venues). Triggers require 2 consecutive positive frames and sustain across an 8-frame hysteresis hold.

IV. DUAL-TIER DEPLOYMENT ARCHITECTURE

A. Cloud Microservice (FastAPI / Render)

Operates within strict 512 MB RAM containers by decoupling PyTorch (`HAS_TORCH=False`), pinning `intra_op_threads=1`, using a zero-memory scene stub (`enable_scene_gating=False`), validating uploads via container magic bytes, enforcing per-IP sliding-window throttling, and persisting incidents to SQLite (WAL) and JSONL.

B. Edge Browser Runtime (Next.js 16 + ONNX Runtime Web)

In Edge Mode, videos stay strictly on the client device. Off-screen decoding enforces `crossOrigin="anonymous"`, `muted`, `playsInline`, `preload="auto"`, pre-flight `canPlayType()` codec validation, native aspect preservation, a 1500 ms seek safeguard, and fault-tolerant `drawImage()` skipping. FastAPI injects `Cross-Origin-Opener-Policy: same-origin`, `Cross-Origin-Embedder-Policy: require-corp`, and `Cross-Origin-Resource-Policy: cross-origin` headers, unlocking `SharedArrayBuffer` 4-thread WASM SIMD (`WASM-SIMD-4T`) and WebGPU acceleration. Native OpenCV cameras run locally >50 FPS.

V. EXPERIMENTAL RESULTS

A. Zero-Leakage Chronological Split (1,951 Videos)

Evaluation uses 1,951 verified clips from *Real Life Violence Situations* [5] partitioned strictly by source ID via a frozen 70/15/15 manifest (`splits_manifest.json`): 1,365 training, 293 validation (sole threshold/gate tuning split), and 293 strictly held-out test clips totaling 42,192 frames (150 Violent, 143 Non-Violent; Train ∩ Test = ∅).

B. Ablation Study & Confusion Matrix Verification

Tables II and III summarize ablation gains and exact confusion matrix derivations on the 293 held-out test clips (TP = 142, FP = 34, FN = 8, TN = 109):

$$\mathrm{Acc} = \frac{142+109}{293} = 85.67\%, \quad \mathrm{Prec} = \frac{142}{176} = 80.68\%$$
$$\mathrm{Rec} = \frac{142}{150} = 94.67\%, \quad \mathrm{Spec} = \frac{109}{143} = 76.22\%, \quad \mathrm{F1} = \frac{284}{326} = 0.871 \quad (13)$$

TABLE II
ABLATION STUDY ON HELD-OUT TEST SET (293 VIDEOS / 42,192 FRAMES)

Configuration	Precision	Recall	Spec.	F1	FP / 143
Baseline ST-GCN	77.72%	**96.60%**	70.63%	0.861	42 (29.4%)
+ Kalman + Strike Gate	79.45%	95.20%	74.13%	0.866	37 (25.9%)
Full (+ Scene Gating)	**80.68%**	**94.67%**	**76.22%**	**0.871**	**34 (23.8%)**

TABLE III
CONFUSION MATRIX ON 293 HELD-OUT TEST VIDEOS

Ground Truth	Pred. Safe	Pred. Violence	Total
Actual Safe (NV)	109 (TN)	34 (FP)	143
Actual Violence (V)	8 (FN)	142 (TP)	150
Total	**117**	**176**	**293**

C. Hardware Latency & Browser WASM Parity

Benchmarking was executed on an Intel Core i7-12700H CPU (16 GB RAM, ONNX Runtime CPU) over **100 timed iterations preceded by a 10-iteration warm-up run**. Table IV reports 16.9 ms summed pure-model latency (59.1 FPS theoretical) and **50.7 FPS** end-to-end throughput (19.7 ms/frame). Automated concatenated regression testing yields 0/66 (0.0%) false alarms and 1.0000 precision, while Node.js `onnxruntime-web` WASM parity testing verifies `peak=0.727` (Violence) on `violence_sample.mp4` and `peak=0.217` (Safe) on `safe_sample.mp4`.

TABLE IV
PER-STAGE LATENCY BREAKDOWN (INTEL CORE I7-12700H, 100 TIMED ITERATIONS AFTER 10 WARM-UP RUNS)

Pipeline Stage	Mean (ms)	Budget / Rate
YOLOv8n-Pose (320×320 ONNX)	11.2	< 20.0 ms
Kalman Occlusion Tracker	0.8	< 2.0 ms
Skeleton Normalization	0.4	< 1.0 ms
ST-GCN Temporal Conv (ONNX)	4.3	< 10.0 ms
Relational Strike Gate	0.2	< 0.5 ms
Summed Model Pipeline Latency	**16.9**	**59.1 FPS**
Measured End-to-End Throughput	**19.7**	**50.7 FPS**

D. Operational Limitations & Failure Mode Analysis

Inspecting misclassified held-out clips reveals clear physical boundaries. First, the 34 residual false positives cluster predominantly in intense athletic sparring, close-quarters wrestling, and choreographed martial-arts routines where fast limb contact satisfies both geometric invariants ($d_{\text{strike}} \leq \alpha \cdot W_{\text{frame}}$ and IoU > 0) despite benign recreational context. When local MobileNetV3 scene classification detects an athletic arena, tightening (α, τ) to (0.12, 0.85) suppresses incidental sports overlap, whereas memory-capped 512 MB cloud deployments disable scene gating (`enable_scene_gating=False`) to preserve container stability.

Second, the 8 false negatives stem from extreme motion blur, severe low-light silhouette degradation, or heavy multi-person overlap that suppresses YOLOv8n-Pose box confidences below detection thresholds prior to skeletal graph evaluation. Finally, browser Edge Mode decoding remains governed by client HTML5 video container codecs (native H.264/AVC1 MP4 and VP8/VP9/AV1 WebM); pre-flight `canPlayType()` verification automatically identifies unsupported codecs such as HEVC/H.265 and directs operators seamlessly to the server-side OpenCV fallback.

VI. CONCLUSION

Edge Vision combines lightweight skeleton-based ST-GCN graph convolutions with an 8-D Kalman occlusion tracker and a pixel-space Relational Strike Gate to achieve **85.67% Accuracy**, **80.68% Precision**, **94.67% Recall**, **76.22% Specificity**, and a **0.871 F1 Score** at **50.7 FPS** on commodity CPUs while enabling zero-upload browser execution via Cross-Origin-Isolated WebAssembly SIMD and WebGPU. Future work will explore INT8 post-training quantization of the pose estimator for ARM SoCs, fine-tuning athletic sparring discrimination, and multi-camera RTSP re-identification.

REFERENCES

[1] D. Tran et al., "Learning spatiotemporal features with 3D convolutional networks," in *Proc. ICCV*, 2015.

[2] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," in *Proc. AAAI*, 2018.

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8 Pose Estimation," 2023. [Online]. Available: https://docs.ultralytics.com/tasks/pose/

[4] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Res. Logist. Quart.*, vol. 2, no. 1–2, pp. 83–97, 1955.

[5] M. M. Soliman et al., "Violence recognition from videos using deep learning techniques," in *Proc. IEEE ICICIS*, 2019.

[6] M. Cheng, K. Cai, and M. Li, "RWF-2000: An open large scale video database for violence detection," in *Proc. ICPR*, 2021.

[7] P. Wu et al., "Not only look, but also listen: Learning multimodal violence detection under weak supervision," in *Proc. ECCV*, 2020.

[8] B. M. Degardin and H. Proença, "Human activity analysis: Iterative weak/self-supervised learning frameworks for detecting abnormal events," in *Proc. IEEE IJCB*, 2020.

[9] M. Gohar, "Violence detection enhancement by involving convolutional block attention modules into various deep learning architectures: Comprehensive case study for UBI-Fights dataset," *IEEE Access*, 2023.

[10] L. Shi et al., "Two-stream adaptive graph convolutional networks for skeleton-based action recognition," in *Proc. CVPR*, 2019.

[11] Z. Liu, H. Zhang, Z. Chen, Z. Wang, and W. Ouyang, "Disentangling and unifying graph convolutions for skeleton-based action recognition," in *Proc. CVPR*, 2020.

[12] Y. Chen et al., "Channel-wise topology refinement graph convolution for skeleton-based action recognition," in *Proc. ICCV*, 2021.

[13] Y.-F. Song, Z. Zhang, C. Shan, and L. Wang, "Constructing stronger and faster baselines for skeleton-based action recognition," *IEEE TPAMI*, 2022.

[14] H. Duan, Y. Zhao, K. Chen, D. Lin, and B. Dai, "Revisiting skeleton-based action recognition," in *Proc. CVPR*, 2022.

[15] H. Duan, J. Wang, K. Chen, and D. Lin, "PYSKL: Towards good practices for skeleton action recognition," in *Proc. ACM MM*, 2022.

[16] H.-g. Chi et al., "InfoGCN: Representation learning for human skeleton-based action recognition," in *Proc. CVPR*, 2022.

[17] J. Lee, M. Lee, D.-L. Nguyen, and S. Lee, "Hierarchically decomposed graph convolutional networks for skeleton-based action recognition," in *Proc. ICCV*, 2023.

[18] J. Do and M. Kim, "SkateFormer: Skeletal-temporal transformer for human action recognition," in *Proc. ECCV*, 2024.

[19] Y. Zhou et al., "BlockGCN: Redefine topology awareness for skeleton-based action recognition," in *Proc. CVPR*, 2024.

[20] H. Yang et al., "Multi-scale and attention enhanced graph convolution network for skeleton-based violence action recognition," *Frontiers in Neurorobotics*, 2022.

[21] N. F. Janbi, M. A. Ghaseb, and A. A. Almazroi, "ESTS-GCN: An ensemble spatial-temporal skeleton-based graph convolutional networks for violence detection," *Int. J. Intell. Syst.*, 2024.

[22] N. Tran, H. Nguyen, D. Ly, and H. D. Nguyen, "Advancing violence detection with graph-based skeleton motion analysis," *SN Computer Science*, 2025.

[23] D. Maji, S. Nagaraj, et al., "YOLO-Pose: Enhancing YOLO for multi person pose estimation," in *Proc. CVPRW*, 2022.

[24] Y. Zhang et al., "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. ECCV*, 2022.

[25] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, "BoT-SORT: Robust associations multi-pedestrian tracking," *arXiv:2206.14651*, 2022.

[26] ONNX Runtime Developers, "ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator," 2024. https://onnxruntime.ai/

[27] A. Howard et al., "Searching for MobileNetV3," in *Proc. ICCV*, 2019.